

CSE 4241: Biomedical Engineering

Assignment 1: Protein Secondary Structure Prediction Using Deep Learning

Submitted To:

Safin Ahmmed

Lecturer

Department of Computer Science and Engineering

Submitted By:

Md Mushfiqur Rahman

Roll: 2007027

Year: 4th

Semester: 2nd



Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

Date: June 22, 2026

Sources

This assignment involves implementing a deep learning pipeline for protein secondary structure prediction. The notebook and dataset links are provided below:

[1] **Training Notebook Repository:**

https://github.com/Mushfiq2007027/Bioinformatics_Assignment/blob/main/2007027-bioinformatics-assignment-1.ipynb

This notebook contains all code for data loading, preprocessing, model design, training, evaluation, and visualization, including weight update GIFs and confusion matrix plots.

[2] **Dataset:**

<https://www.kaggle.com/datasets/alfrandom/protein-secondary-structure>

This dataset contains protein sequences with 3-state (H/E/C) and 8-state secondary structure annotations derived from the Protein Data Bank (PDB), pre-cleaned and filtered for quality. It contains 393,732 entries across seven fields: `pdbid`, `chaincode`, `seq`, `sst8`, `sst3`, `len`, and `has_nonstd_aa`.

Contents

Sources	1
1 Introduction	4
1.1 Significance of Protein Secondary Structure Prediction	4
1.2 Motivation for a Deep Learning Approach	5
2 Methodology	6
2.1 Dataset	6
2.1.1 Source and Format	6
2.1.2 Filtering and Subsampling	6
2.2 Preprocessing	7
2.2.1 Amino Acid Encoding	7
2.2.2 Secondary Structure Label Encoding	7
2.2.3 Sequence Padding and Truncation	7
2.2.4 One-Hot Encoding of Labels	7
2.2.5 Train/Validation/Test Split	8
2.3 Model Architecture	8
2.3.1 Layer Description	8
2.3.2 Design Rationale	10
2.4 Training Procedure	11
2.4.1 Loss Function and Optimizer	11
2.4.2 Callbacks	11
2.4.3 Training Configuration	11
3 Results	13
3.1 Training and Validation Curves	13
3.1.1 Interpretation of Learning Dynamics	13
3.2 Per-Class Evaluation Metrics	14
3.2.1 Per-Class Analysis	14
3.3 Confusion Matrix	15
3.3.1 Confusion Matrix Analysis	15
3.4 Weight Update Visualisation	16
3.4.1 BiLSTM Layer Weight Evolution	16
3.4.2 Dense Layer Weight Evolution	16

4	Discussion	17
4.1	Model Performance in Context	17
4.2	Sheet Prediction Difficulty	17
4.3	Potential Improvements	18
4.4	Observations on Weight Dynamics	19
5	Conclusion	20

1 Introduction

Proteins are the fundamental molecular machines of all living organisms. They carry out virtually every biological function — catalysing chemical reactions as enzymes, transmitting signals between cells, defending against pathogens as antibodies, and providing structural support to tissues. Despite this staggering functional diversity, all proteins are built from the same set of twenty standard amino acids arranged in linear chains called **primary sequences**. Understanding the relationship between a protein’s primary amino acid sequence and its three-dimensional structure is one of the central challenges of structural biology and computational bioinformatics.

Protein structure is described at four levels. The **primary structure** is the linear sequence of amino acids. The **secondary structure** describes locally recurring patterns in this sequence — most importantly, alpha-helices (H), beta-strands (E), and irregular coils or loops (C). The **tertiary structure** is the complete three-dimensional fold of the protein chain, and the **quaternary structure** describes how multiple chains assemble. Secondary structure is the most tractable of these to predict computationally, and it serves as a crucial intermediate representation that simplifies the full tertiary structure prediction problem.

Protein secondary structure prediction (PSSP) is the task of assigning each residue in a protein sequence to one of a fixed number of structural classes. In the standard three-class formulation (Q3 prediction), each residue is labelled as belonging to a helix (H), a beta-sheet strand (E), or a coil/loop (C). The eight-class formulation (Q8) is more fine-grained, distinguishing different types of helices, strands, and turns, but Q3 remains the most widely studied benchmark.

1.1 Significance of Protein Secondary Structure Prediction

The importance of PSSP extends across multiple scientific and medical domains:

1. **Drug Discovery:** Many drugs work by binding to specific structural motifs of proteins. Knowing which regions of a target protein form helices or strands helps medicinal chemists design molecules that fit precisely into the protein’s active site. Errors in structure prediction can lead to costly failed drug trials.
2. **Understanding Disease:** Mutations that alter secondary structure are responsible for a wide range of diseases. For example, amyloid beta-sheets are implicated in Alzheimer’s disease, and misfolding of the prion protein (converting alpha-helices to beta-sheets) causes prion diseases such as Creutzfeldt-Jakob disease.
3. **Protein Engineering:** Synthetic biologists and protein engineers design novel proteins for biotechnology applications — biosensors, industrial enzymes, therapeutic

scaffolds. PSSP allows in-silico validation that a designed sequence will adopt the intended fold before expensive wet-lab synthesis.

4. **Comparative Genomics:** Secondary structure is more conserved across evolution than primary sequence. Two proteins with only 20% sequence identity may have nearly identical secondary structures. PSSP therefore enables structural comparisons between distantly related proteins.
5. **Foundation for 3D Structure Prediction:** Tools like AlphaFold2 and RoseTTAFold implicitly predict secondary structure as an intermediate step in full 3D structure prediction. Understanding PSSP is therefore foundational to understanding modern structural biology AI.

1.2 Motivation for a Deep Learning Approach

Early PSSP methods such as Chou-Fasman rules and GOR (Garnier-Osguthorpe-Robson) achieved Q3 accuracies of around 60–65% by exploiting the statistical propensities of individual amino acids to form each type of structure. These methods were limited because secondary structure is a *context-dependent* property — whether a residue forms a helix or a strand depends not just on its own identity, but on the identities of the residues around it, and on long-range interactions spanning tens to hundreds of positions.

Hidden Markov models and neural networks brought accuracy into the 70–75% range by incorporating local sequence windows. Modern deep learning architectures — recurrent neural networks, convolutional networks, and attention-based transformers — can learn extremely long-range dependencies and have pushed state-of-the-art Q3 accuracy above 85%. Crucially, deep models learn these dependencies *automatically* from data, without requiring manually engineered features.

This assignment implements a deep learning PSSP pipeline from scratch, using a Bidirectional Long Short-Term Memory (BiLSTM) network. The goal is to reproduce the key design decisions of modern PSSP systems at a tractable scale, to understand the preprocessing, modelling, training, and evaluation pipeline end-to-end, and to visualize how the network learns over the course of training.

2 Methodology

2.1 Dataset

2.1.1 Source and Format

The dataset used in this assignment is the *Protein Secondary Structure* dataset published on Kaggle by AlFreedom, derived from the Protein Data Bank (PDB). The primary file used is `2018-06-06-ss.cleaned.csv`, which contains 393,732 protein chain entries. Each entry has seven fields:

- `pdbid`: The four-character PDB identifier of the protein structure.
- `chaincode`: The chain identifier (e.g., A, B, C).
- `seq`: The amino acid sequence as a string of single-letter codes.
- `sst8`: The eight-state secondary structure annotation (one character per residue).
- `sst3`: The three-state secondary structure annotation (H, E, or C per residue).
- `len`: The length of the sequence in residues.
- `has_nonstd_aa`: Boolean flag indicating whether the sequence contains non-standard amino acids.

2.1.2 Filtering and Subsampling

The raw dataset was filtered with the following criteria before training:

1. **Remove non-standard amino acid sequences:** Entries with `has_nonstd_aa = True` were dropped. Non-standard amino acids (e.g., selenocysteine, pyrrolysine) cannot be encoded by the standard 20-letter vocabulary and would introduce unknown tokens.
2. **Length filtering:** Sequences shorter than 50 residues or longer than 500 residues were removed. Very short sequences are not representative of typical protein folds. Very long sequences would require extremely long padded tensors, greatly increasing memory and computation requirements.
3. **Subsampling:** After filtering, 333,791 sequences remained. To keep the total training time within approximately three hours on the Kaggle CPU environment, a random subset of 60,000 sequences was sampled using a fixed random seed (`random_state=42`) for reproducibility.

After subsampling, the dataset statistics were: 60,000 sequences, sequence lengths ranging from 50 to 500 residues, and an average sequence length of approximately 232 residues.

2.2 Preprocessing

2.2.1 Amino Acid Encoding

The twenty standard amino acids (A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y) were mapped to integer indices 1 through 20. Index 0 was reserved as the padding token. Each protein sequence was converted from its string representation into a list of integers using this mapping. Any character not in the standard alphabet was mapped to 0 (treated as padding).

2.2.2 Secondary Structure Label Encoding

The three-state secondary structure labels were encoded as follows:

- H (helix) $\rightarrow$ 0
- E (strand) $\rightarrow$ 1
- C (coil/loop) $\rightarrow$ 2

Any unknown characters in the `ss_t3` field were mapped to 2 (coil) by default, which is the most common class and the least harmful default assignment.

2.2.3 Sequence Padding and Truncation

All sequences were padded or truncated to a fixed maximum length of `MAX_LEN = 300` residues using post-padding (padding appended at the end) and post-truncation (long sequences are cut off at the end). Padding tokens received integer value 0 in the input sequences. Padding positions in the label sequences received the coil label (2), which were later masked out during evaluation.

2.2.4 One-Hot Encoding of Labels

The integer label sequences were converted to one-hot encoded tensors using `tf.keras.utils.to_categorical` with `num_classes=3`. Each label position became a vector $[1, 0, 0]$, $[0, 1, 0]$, or $[0, 0, 1]$ corresponding to H, E, or C respectively. The final shape of the label tensor was (60000, 300, 3).

2.2.5 Train/Validation/Test Split

The 60,000 samples were split into three non-overlapping sets:

Table 1: Dataset Split Statistics

Split	Samples	Fraction
Training	41,820	69.7%
Validation	9,180	15.3%
Test	9,000	15.0%
Total	60,000	100 %

The split was performed in two stages: first, 15% was held out as the test set; then 18% of the remaining data was held out as the validation set. All splits used `random_state=42` for reproducibility.

2.3 Model Architecture

The model is a sequence-to-sequence architecture that takes a padded protein sequence of length 300 as input and produces a secondary structure label for each residue position. The architecture is named **ProteinSSSimple**.

2.3.1 Layer Description

The full architecture is described in Table 2 and illustrated in Figure 1.

Table 2: Model Architecture — Layer-by-Layer Summary

Layer Name	Type	Description	Parameters
sequence_input	InputLayer	Accepts integer-encoded sequences of shape (None, 300). Serves as the entry point for both the embedding branch and the masking branch.	0
amino_acid_embedding	Embedding	Maps each integer token (1–20) to a dense 32-dimensional vector. <code>mask_zero=True</code> passes a padding mask downstream so recurrent layers ignore pad positions. Input shape: (None, 300); Output shape: (None, 300, 32).	672
not_equal_2	NotEqual	Computes a boolean mask by checking which positions in the input are not equal to 0 (the padding token). Output shape: (None, 300). This mask is fed directly into the BiLSTM layer to prevent gradients from flowing through padded positions.	0
bilstm1	Bidirectional (LSTM)	A bidirectional LSTM with 64 units in each direction (128 total). <code>return_sequences=True</code> so that an output vector is produced for every input timestep, enabling per-residue prediction. Dropout rate 0.2, recurrent dropout 0.1. Input: [(None,300,32),(None,300)]; Output: (None, 300, 128).	49,664
dense1	Dense	A point-wise fully connected layer with 32 units and ReLU activation applied independently at each timestep. Reduces the 128-dimensional BiLSTM output to a compact 32-dimensional representation. Input: (None, 300, 128); Output: (None, 300, 32).	4,128
drop1	Dropout	Applies dropout with rate 0.3 during training to regularize the dense layer output and prevent overfitting. Input/Output shape: (None, 300, 32).	0
output	Dense	Final classification layer with 3 units and softmax activation. Produces a probability distribution over the 3 classes (H, E, C).	99

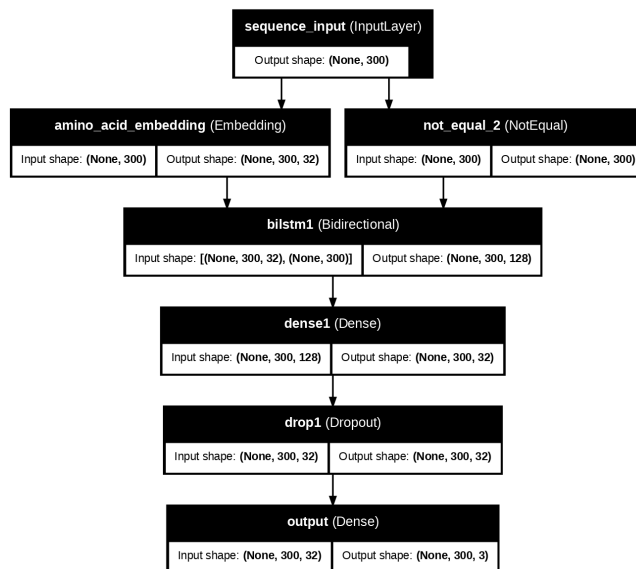


Figure 1: Model architecture diagram of ProteinSSSimple. The input layer branches into an embedding layer and a masking layer, whose outputs are both consumed by the BiLSTM. The BiLSTM output is passed through a dense layer, dropout, and a final softmax output layer.

2.3.2 Design Rationale

Bidirectional LSTM: Secondary structure at a given residue depends on the context of amino acids both before and after it in the sequence. A unidirectional LSTM only sees past context; a BiLSTM processes the sequence in both the forward and backward directions and concatenates the two hidden states, effectively giving every position access to its full sequence context. This is the standard architectural choice for per-residue sequence labelling tasks.

Embedding Layer with Masking: Rather than one-hot encoding the input (which would produce a sparse 20-dimensional vector for each residue), a learned 32-dimensional embedding is used. This allows the model to discover a dense, geometrically meaningful representation of amino acids where chemically similar residues map to nearby vectors. The `mask_zero=True` argument ensures that padding positions are ignored throughout the network.

Moderate Capacity: With only 54,563 parameters, the model is deliberately kept small. This prevents severe overfitting on the 60,000-sequence training set and keeps the per-epoch training time within the approximately 9–10 minute range achievable on a CPU-only Kaggle notebook.

Dropout for Regularization: A 0.3 dropout rate is applied before the final output layer to prevent the dense layer from memorizing training examples. A 0.2 dropout is also applied inside the LSTM along the input-to-hidden connections, and a 0.1 recurrent dropout is applied along the hidden-to-hidden connections.

2.4 Training Procedure

2.4.1 Loss Function and Optimizer

The model was compiled with:

- **Loss:** Categorical cross-entropy. For a batch of sequences, the loss is computed only over non-padding positions (the masking layer handles this implicitly via the `mask_zero` mechanism in Keras).
- **Optimizer:** Adam with an initial learning rate of $\eta = 1 \times 10^{-3}$.
- **Metric:** Categorical accuracy (fraction of non-padding residue positions correctly classified).

2.4.2 Callbacks

Three callbacks were used during training:

1. **EarlyStopping:** Monitored validation loss with a patience of 5 epochs. If the validation loss failed to improve for 5 consecutive epochs, training was halted and the weights from the best epoch were restored. This prevents wasting compute on epochs that do not improve generalization.
2. **ReduceLROnPlateau:** Reduced the learning rate by a factor of 0.5 whenever the validation loss failed to improve for 3 consecutive epochs, with a minimum learning rate of 1×10^{-6} . This allows the optimizer to take smaller, more precise steps when it approaches a minimum.
3. **ModelCheckpoint:** Saved the model weights to `best_model.h5` whenever the validation loss improved, ensuring the best checkpoint was preserved even if training continued for additional epochs before early stopping triggered.
4. **WeightTracker (Custom Callback):** A custom `tf.keras.callbacks.Callback` that, at the end of each epoch, extracted and stored a copy of the kernel (weight matrix) of the `bilstm1` and `dense1` layers. These stored weights were later used to generate GIF animations of weight evolution over training epochs.

2.4.3 Training Configuration

- **Maximum epochs:** 20
- **Batch size:** 128 sequences per step

- **Steps per epoch:** 327 ($= \lceil 41820/128 \rceil$)
- **Environment:** Kaggle CPU-only notebook, TensorFlow 2.19.0, Python 3.12.13
- **Total training time:** Approximately 3 hours for 20 epochs

3 Results

3.1 Training and Validation Curves

Figure 2 shows the training and validation loss and accuracy curves over 20 epochs of training.



Figure 2: Training and validation metrics over 20 epochs. Left: Loss curves for training (solid blue) and validation (dashed orange). Both curves decrease monotonically throughout training. The validation loss is consistently lower than the training loss, indicating that the model generalises well without overfitting. Right: Accuracy curves for training (solid blue) and validation (dashed orange). Validation accuracy is slightly higher than training accuracy throughout, consistent with the dropout regularisation being active only during training.

3.1.1 Interpretation of Learning Dynamics

Several important observations can be drawn from the training curves:

1. **Rapid early improvement:** Both loss and accuracy improve most sharply in the first 1–5 epochs. The model learns the dominant structural patterns (rough frequency of each class) very quickly.
2. **Diminishing returns:** Improvement slows progressively from epoch 5 onwards. This is typical of gradient descent on a non-convex landscape: large, easily correctable errors are fixed first, and smaller refinements require more epochs.
3. **No overfitting:** The validation loss remains consistently *below* the training loss, and validation accuracy remains above training accuracy. This somewhat counter-intuitive pattern occurs because dropout regularisation is active during training (increasing training loss) but inactive during evaluation. It indicates that the model is not memorising the training data.

4. **Stable convergence:** By epoch 20, both training and validation curves have plateaued, suggesting the model has effectively reached a local minimum with the current learning rate schedule.

The training loss decreased from 0.8520 at epoch 1 to 0.6549 at epoch 20, and the validation loss decreased from 0.7789 to 0.6109. Training accuracy improved from 0.6160 to 0.7219, and validation accuracy improved from 0.6550 to 0.7449 over the same period.

3.2 Per-Class Evaluation Metrics

Table 3 presents precision, recall, F1-score, and support for each class on the masked test set (padding positions excluded from all calculations).

Table 3: Per-Class Evaluation Metrics on the Test Set (Masked)

Class	Precision	Recall	F1-Score	Support
Helix (H)	0.7663	0.7779	0.7720	631,832
Sheet (E)	0.7171	0.6240	0.6673	422,220
Coil (C)	0.7257	0.7652	0.7449	831,570
Macro Avg	0.7363	0.7223	0.7281	1,885,622

Overall residue-level accuracy on the test set: 73.78%

3.2.1 Per-Class Analysis

Helix (H): Helices achieved the highest F1-score of 0.7720. This is expected: alpha-helices are characterised by a very regular pattern of hydrogen bonding and tend to involve amino acids with high helix-forming propensities (alanine, leucine, glutamate, methionine, lysine). These regularities make them the most learnable structural class. The recall of 0.7779 indicates that the model correctly identifies about 78% of all true helix residues.

Sheet (E): Beta-sheets achieved the lowest F1-score of 0.6673, driven primarily by a lower recall of 0.6240. This means the model missed approximately 38% of true strand residues, often labelling them as coils. Beta-sheets are significantly more difficult to predict because: (a) they frequently involve non-local interactions between residues far apart in sequence, (b) the strands themselves are short and separated by loops, and (c) their amino acid propensities are less sharply defined than those of helices.

Coil (C): Coil residues had a recall of 0.7652 and a precision of 0.7257. This reflects a common pattern: because coil is the most frequent class (831,570 test residues, compared to 631,832 for helix and 422,220 for sheet), models tend to fall back to predicting coil when uncertain, inflating coil recall at the cost of sheet recall.

3.3 Confusion Matrix

Figure 3 shows the confusion matrix on the full test set in both raw counts and row-normalised form.

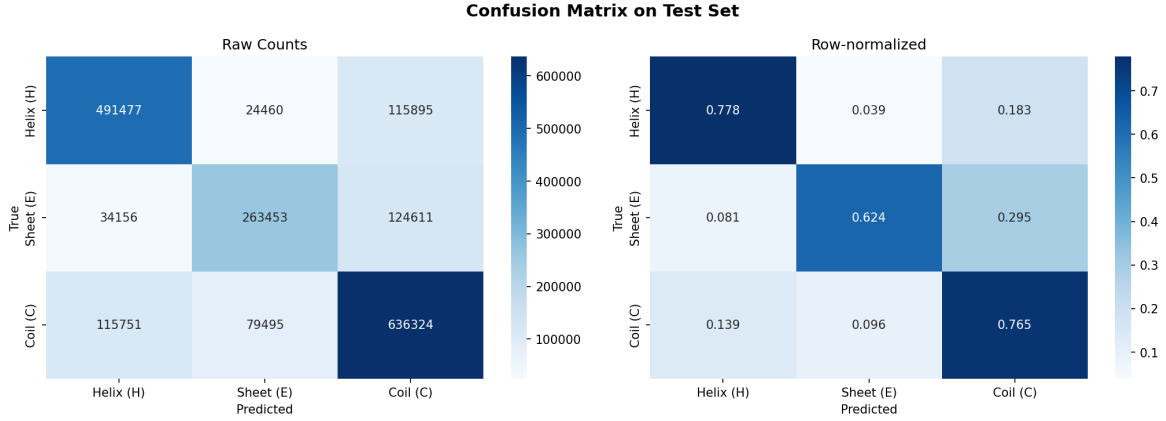


Figure 3: Confusion matrix on the test set. Left: Raw residue counts. Right: Row-normalised matrix showing the fraction of true residues in each class assigned to each predicted class. The diagonal entries represent correct predictions; off-diagonal entries represent misclassifications.

3.3.1 Confusion Matrix Analysis

The row-normalised confusion matrix reveals the following classification patterns:

- **Helix (H):** 77.8% of true helix residues are correctly classified as helix. 3.9% are confused with sheets, and 18.3% are confused with coils. The H→C confusion is the dominant error mode for helices.
- **Sheet (E):** Only 62.4% of true strand residues are correctly classified. 8.1% are confused with helices, and 29.5% are confused with coils. The E→C confusion accounts for almost a third of all strand residues, confirming that the model frequently labels uncertain strands as coils.
- **Coil (C):** 76.5% of true coil residues are correctly classified. 13.9% are confused with helices, and 9.6% are confused with strands. The C→H confusion direction is notable and may reflect short coil turns that have helix-like local sequence environments.

The overall confusion pattern — particularly the asymmetry between sheet recall and helix recall — is consistent with published results in the PSSP literature and reflects the intrinsic difficulty of predicting beta-sheet structure from local sequence context alone.

3.4 Weight Update Visualisation

A custom `WeightTracker` callback recorded the kernel weight matrix of the `bi_lstm1` and `dense1` layers at the end of each training epoch. GIF animations were generated to visualise how these weights evolved over the 20 epochs of training.

3.4.1 BiLSTM Layer Weight Evolution

The BiLSTM kernel has input shape (32, 256) — a 32-dimensional embedding input mapped to 4 concatenated gate matrices (input, forget, cell, output), each with 64 units per direction, giving 256 columns. In epoch 1, the weight heatmap is dominated by small, noisy values with a slight positive (reddish) bias, indicating that the random initialisation has not yet produced meaningful structure. By the final epoch, clear column-wise and row-wise patterns emerge, reflecting that the input gate and forget gate weights have organised themselves to respond preferentially to specific amino acid embedding dimensions.

3.4.2 Dense Layer Weight Evolution

The `dense1` kernel has shape (128, 32) — a 128-dimensional BiLSTM output mapped to 32 hidden units. The initial weight matrix shows a balanced mixture of positive (red) and negative (blue) values with moderate magnitude. Over training, the weights organise into a striped pattern where certain output units develop strong positive or negative responses to specific subsets of the BiLSTM dimensions, reflecting the emergence of interpretable feature detectors for structural classes.

4 Discussion

4.1 Model Performance in Context

The achieved test accuracy of 73.78% Q3 and macro F1-score of 0.7281 are competitive for a lightweight model trained on 60,000 sequences for 20 epochs without any pre-trained protein language model embeddings. For context:

- Early statistical methods (GOR, Chou-Fasman) achieved Q3 accuracies of 60–65%.
- The introduction of neural networks in the 1990s (Qian and Sejnowski, 1988) raised accuracy to approximately 64–66%.
- Profile-based approaches using evolutionary information (PSI-BLAST + neural networks) pushed Q3 accuracy above 75%.
- State-of-the-art deep learning systems incorporating pre-trained protein language models such as ESM-1b and ProtBERT have exceeded 85% Q3 accuracy.

The ProteinSSSimple model falls between the classical neural network era and the profile-based era in terms of absolute accuracy. This is expected given its simple architecture, the absence of evolutionary information (position-specific scoring matrices), and the restriction to 60,000 training sequences. The gap to state-of-the-art systems is attributable primarily to three factors:

1. **No evolutionary information:** Modern PSSP systems use multiple sequence alignments (MSAs) to capture conservation patterns across hundreds of homologous sequences. These patterns are far more informative than the raw sequence alone.
2. **No pre-trained embeddings:** Protein language models like ESM-2, trained on hundreds of millions of protein sequences, produce embeddings that already encode rich structural knowledge. Using these as features rather than training an embedding from scratch on 60,000 sequences would substantially improve accuracy.
3. **Limited model depth:** A single BiLSTM layer can capture local and medium-range context, but cannot capture the long-range interactions that determine beta-sheet pairing between distant strands.

4.2 Sheet Prediction Difficulty

The most striking result is the large performance gap between helix prediction ($F1 = 0.7720$) and sheet prediction ($F1 = 0.6673$). This gap arises from a fundamental structural difference between the two types:

Alpha-helices are determined primarily by **local sequence preferences**. A stretch of 7–10 consecutive helix-forming amino acids produces a helix that can be identified from the sequence alone. Beta-sheets, by contrast, are formed by **non-local hydrogen bonds** between strands that may be separated by hundreds of residues in the linear sequence. A model that only processes the local sequence context around each residue (as a single-layer BiLSTM effectively does, despite seeing the full sequence) will systematically under-predict strand residues.

The high E→C confusion rate (29.5%) also suggests that when the model is uncertain, it defaults to coil. This is partly a consequence of class imbalance (coil is the most frequent class) and partly because the coil category is a catch-all for all non-regular structure — it is the structurally most heterogeneous class, making it a reasonable default for uncertain predictions.

4.3 Potential Improvements

Based on the analysis above, the following improvements would be expected to increase model performance:

1. **Add a second BiLSTM layer:** A two-layer BiLSTM increases the effective context window and allows the model to learn hierarchical representations of sequence patterns.
2. **Incorporate a convolutional layer before the BiLSTM:** A 1D convolutional layer with multiple filter widths (e.g., 3, 5, 7 residues) can extract local n -gram features before the BiLSTM processes them as a sequence.
3. **Use the full dataset:** Training on all 333,791 filtered sequences (rather than 60,000) would give the model substantially more diverse structural examples, particularly for rarer secondary structure elements.
4. **Incorporate positional information:** Adding a position embedding or sinusoidal positional encoding allows the model to account for position-specific structural biases (e.g., terminal residues are more often coil).
5. **Apply class-weighted loss:** Although the masking handles padding positions, the class imbalance between H (33.5%), E (22.4%), and C (44.1%) could be corrected by weighting the loss inversely to class frequency. This would particularly benefit sheet prediction.
6. **Use pre-trained amino acid embeddings:** Replacing the randomly initialised embedding with embeddings from a protein language model (e.g., ESM-2 with 6M, 35M,

or 150M parameters) would encode evolutionary and biophysical information that is not learnable from sequence alone.

7. **Add an attention mechanism:** A self-attention layer after the BiLSTM would allow the model to directly attend to specific long-range positions relevant to each residue's structure prediction, directly addressing the non-local nature of beta-sheet formation.
8. **Use Transformer architecture:** Replacing the BiLSTM entirely with a multi-head self-attention Transformer encoder would give $O(1)$ effective distance between any two positions in the sequence, substantially improving sheet prediction.

4.4 Observations on Weight Dynamics

The weight update visualisations reveal a consistent pattern across both layers: the initial random weights are noisy and unstructured, and meaningful patterns emerge primarily within the first 5–8 epochs, consistent with the rapid early improvement seen in the loss and accuracy curves. After epoch 10, weight changes become much smaller — the GIF animations show subtler colour shifts — corresponding to the diminishing returns phase observed in the training curves.

The BiLSTM weights show stronger horizontal banding (input feature selectivity) than vertical banding (output unit selectivity), suggesting that the model's primary learned inductive bias is which amino acid embedding dimensions to amplify, rather than which output hidden units to activate. The dense1 layer shows the opposite pattern: clear vertical structure suggesting that different output units have specialised for different structural class predictions.

5 Conclusion

This assignment successfully implemented a complete deep learning pipeline for protein secondary structure prediction, from raw sequence data through a Bidirectional LSTM model to per-residue structural class assignment. The model achieved a Q3 accuracy of 73.78% and a macro F1-score of 0.7281 on the held-out test set, with strongest performance on helix prediction ($F1 = 0.7720$) and weakest on sheet prediction ($F1 = 0.6673$).

The results validate the effectiveness of bidirectional recurrent architectures for sequence-to-sequence biological prediction tasks, while also clearly illustrating the limitations of simple single-layer BiLSTM models — particularly for structural classes like beta-sheets that are determined by non-local interactions. The confusion matrix and per-class metrics provide an informative diagnostic of where model improvements should be focused.

Weight update GIF animations demonstrated that the BiLSTM and dense layer weights evolve most rapidly in early training epochs and gradually stabilise, consistent with the learning curves showing rapid early improvement followed by diminishing returns. These visualisations provide an intuitive window into the learning dynamics of the trained network. The overall pipeline — data filtering, integer encoding, padding, BiLSTM modelling, dropout regularisation, callback-based training control, and masked evaluation — is representative of modern practice for sequence labelling tasks in bioinformatics and serves as a foundation for more advanced architectures incorporating evolutionary information, pre-trained protein language model embeddings, and attention mechanisms.